



ABLESTACK Online Docs  
ABLESTACK-V4.0-4.0.15

# 계정 및 사용자 관 리

# 계정 및 사용자 관리

ABLESTACK은 Cloud 서비스 위하여 도메인과 계정을 통하여 자원을 구분하고 권한 및 역할에 따라서 사용자를 구분할 수 있는 관리 기능을 제공합니다.

## 역할(Roles), 계정(Account), 도메인(Domain), 사용자(User)

### 역할(Roles)

역할은 허용된 기능 집합을 나타냅니다. 모든 ABLESTACK 계정에는 API 요청을 허용하거나 허용하지 않도록 액세스 규칙을 적용하는 역할이 연결되어 있습니다. 일반적으로 루트 관리자, 리소스 관리자, 도메인 관리자 및 사용자의 네 가지 기본 역할이 있습니다.

### 계정(Account)

계정은 일반적으로 서비스 제공 업체의 고객 또는 대규모 조직의 부서를 나타냅니다. 한 계정에 여러 사용자가 존재할 수 있습니다.

### 도메인(Domain)

계정은 도메인별로 그룹화되며 일반적으로 도메인에는 서로 논리적 관계가 있는 여러 계정과 도메인 및 하위 도메인에 대한 권한이 있는 위임된 관리자 집합이 포함됩니다. 예를 들어 여러 리셀러가 있는 서비스 공급자는 각 리셀러에 대한 도메인을 만들 수 있습니다.

최고 관리자 유형의 계정 (최상위 도메인에서만 사용 가능) 외에 도메인 관리자와 사용자라는 두 가지 유형의 계정을 각 도메인에 만들 수 있습니다.

### 사용자(User)

사용자는 계정의 별칭과 같으며 동일한 계정의 사용자는 서로 격리되지 않지만 다른 계정의 사용자와 격리됩니다.

사용자 이름은 해당 도메인의 계정 전체에서 중복될 수 없으며 하위 도메인을 포함한 다른 도메인에서는 존재할 수 있습니다.

### 도메인 관리자(Domain Administrators)

도메인 관리자는 해당 도메인에 속한 사용자에게 대한 관리 작업을 수행할 수 있습니다. 도메인 관리자는 물리적 서버 나 다른 도메인을 볼 수 없습니다.

### 최고 관리자(Root Administrator)

최고 관리자는 템플릿, 서비스 제공, 고객 관리 관리자 및 도메인 관리를 포함하여 시스템에 대한 완전한 액세스 권한을 갖습니다.

자원에 대한 소유권(Ownership) 자원은 해당 계정의 개별 사용자가 아니라 계정에 속합니다. 예를 들어 청구, 자원 제한 등은 사용자가 아닌 계정에서 유지 관리됩니다. 사용자는 해당 작업에 대한 권한이 있는 경우 계정의 모든 자원에서 작업할 수 있습니다. 권한은 역할에 따라 결정됩니다. 최고 관리자는 가상 머신의 소유권을 한 계정에서 다른 계정으로 변경할 수 있습니다. 도메인 또

는 하위 도메인 관리자는 한 계정에서 도메인 또는 하위 도메인의 다른 계정으로 도메인 내의 VM에 대해 동일한 작업을 수행할 수 있습니다.

## 동적 역할 사용

기본 역할 외에도 동적 역할 기반 API 검사기 기능을 통해 Mold 루트 관리자는 사용자 지정 권한으로 새 역할을 만들 수 있습니다. 허용/거부 규칙은 관리 서버를 다시 시작하지 않고도 런타임 중에 동적으로 구성할 수 있습니다.

모든 역할은 관리자, 리소스 관리자, 도메인 관리자 및 사용자의 네 가지 역할 유형 중 하나로 확인됩니다. UI의 역할 탭을 사용하고 이름, 역할 유형 또는 기존 역할의 ID, 선택적으로 설명을 지정하여 새 역할을 작성할 수 있습니다. 기존 역할의 ID를 사용하여 새 역할을 생성하면 기존 역할의 모든 규칙이 새 역할에 복사되고 이러한 규칙은 원하대로 수정할 수 있습니다.

역할별 규칙은 역할별 세부 정보 페이지의 규칙 탭을 통해 구성하거나 역할 유형으로 새 역할을 생성하는 동안 CSV 파일에서 가져올 수 있습니다. 규칙은 허용 또는 거부 권한 중 하나이며 선택적으로 설명인 API 이름 또는 와일드 카드 문자열입니다. 이러한 규칙은 CSV 파일로 내보낼 수 있으며 이름은 기본적으로 "\_csv"로 지정됩니다.

CSV 파일 형식 :

```
rule,permission,description
<Rule1>,<Permission1>,<Description1>
<Rule2>,<Permission2>,<Description2>
<Rule3>,<Permission3>,<Description3>
...
so on
```

사용자가 API를 요청하면 백엔드는 요청된 API를 호출자 사용자 계정의 역할에 대해 구성된 규칙 (규칙이 구성된 순서대로)과 비교하여 확인합니다. 규칙을 반복하고 API가 허용 규칙과 일치하면 API 요청을 허용하고 거부 규칙과 일치하면 요청을 거부합니다. 다음으로 요청 API가 구성된 규칙과 일치하지 않는 경우 요청된 API의 기본 권한 부여가 해당 사용자 역할 유형을 허용하는 경우 허용하고 역할 권한 규칙에 의해 명시적으로 허용/거부되지 않으면 최종적으로 사용자 API 요청을 거부합니다. 또는 기본 API가 주석을 승인합니다.

### Info

루트 관리자가 시스템에서 잠기는 것을 방지하기 위해 모든 루트 관리자 계정에 모든 API가 허용됩니다.

동적 역할 기능은 Mold 설치 시 기본적으로 활성화됩니다. 관리자가 마이그레이션 도구를 실행하여 이 기능을 사용하도록 기존 배포를 마이그레이션할 수 있습니다. 마이그레이션 도구가 있습니다.

```
/usr/share/cloudstack-common/scripts/util/migrate-dynamicroles.py.
```

**참고 :** 언제든지 **commands.properties** 파일을 변경하지 않은 경우 **-D (기본값)** 옵션을 사용하는 것이 좋습니다. 그렇지 않으면 새 API 명령이 동적 역할 데이터베이스에 추가되지 않을 수 있습니다.

마이그레이션 중에 이 도구는 데이터베이스에서 내부 플래그를 활성화하고 제공된 **commands.properties** 파일 (일반적으로)에서 데이터베이스로 기존 정적 역할 기반 규칙을 복사 `/etc/cloudstack/management/commands.properties` 하고 **commands.properties** 파일 (일반적으로 `/etc/cloudstack/management/`)의 이름을 바꿉니다. **commands.properties.deprecated**). 마이그레이션 프로세스에서는 관리 서버를 다시 시작할 필요가 없습니다.

사용법 : `migrate-dynamicroles.py[옵션] [-h for help]`

옵션 :

| 옵션             | 내용                                                                              |
|----------------|---------------------------------------------------------------------------------|
| -b DB          | 데이터베이스 이름, 기본값 : 클라우드                                                           |
| -u USER        | 사용자 이름은 클라우드 데이터베이스에 대한 권한이 있는 MySQL 사용자입니다. 기본값 : cloud                        |
| -p<br>PASSWORD | 클라우드 데이터베이스에 대한 권한이 있는 MySQL 사용자의 비밀번호                                          |
| -H HOST        | MySQL 서버의 호스트 또는 IP                                                             |
| -P PORT        | MySQL 서버의 호스트 또는 IP, 기본값 : 3306                                                 |
| -f FILE        | commands.properties 파일, 기본값 :<br>/etc/cloudstack/management/commands.properties |
| -d             | 이 도구가 수행할 테스트 실행 및 디버그 작업                                                       |
| -D             | 동적 역할에 대한 기본 구성 사용 (command.properties를 가져 오지 않음)                               |

예 :

```
sudo python /usr/share/cloudstack-common/scripts/util/migrate-dynamicroles.py -u cloud -p cloud  
-H localhost -P 3306 -f /etc/cloudstack/management/commands.properties  
  
sudo python /usr/share/cloudstack-common/scripts/util/migrate-dynamicroles.py -u cloud -p cloud  
-H localhost -P 3306 -D
```

여러 관리 서버가 있는 경우 첫 번째 관리 서버에 대한 마이그레이션 도구를 실행한 후 일반적으로  
/etc/cloudstack/management 경로에 있는 모든 관리 서버에서 commands.properties 파일을 제거하거나 이름을 바꿉니다.

## 계정 및 도메인 전용 리소스

루트 관리자는 추가 보안 또는 성능 보장을 위해 개인 인프라가 필요한 특정 도메인 또는 계정에 리소스를 할당할 수 있습니다.  
zone, pod, cluster 또는 호스트는 특정 도메인 또는 계정에 대해 루트 관리자가 예약할 수 있습니다. 해당 도메인 또는 하위 도메인의 사용자만 인프라를 사용할 수 있습니다. 예를 들어 지정된 도메인의 사용자만 해당 도메인 전용 zone에 게스트를 만들 수 있습니다.

사용 가능한 유형의 dedication :

- Explicit dedication. zone, pod, cluster 또는 호스트는 초기 배포 및 구성 중에 루트 관리자가 계정 또는 도메인 전용으로 사용합니다.
- Strict implicit dedication. 호스트는 여러 계정에서 공유되지 않습니다. 예를 들어, 엄격한 암시적 전용은 데스크톱 소프트웨어의 라이선스 약관을 위반하지 않고 다른 계정 간에 호스트를 공유할 수 없는 데스크톱과 같은 특정 유형의 애플리케이션 배포에 유용합니다.
- Preferred implicit dedication. 가능한 경우 VM은 전용 인프라에 배포됩니다. 그렇지 않으면 VM을 공유 인프라에 배포할 수 있습니다.

## zone, pod, cluster 또는 호스트를 계정 또는 도메인 전용으로 지정하는 방법

명시적 전용 : 새 zone, pod, cluster 또는 호스트를 배포할 때 루트 관리자는 전용 확인란을 클릭한 다음 리소스를 소유할 도메인 또는 계정을 선택할 수 있습니다.

기존 zone, pod, cluster 또는 호스트를 명시적으로 전용 : 루트 관리자로 로그인하고 UI에서 리소스를 찾은 다음 전용 버튼을 클릭합니다.

암시적 헌신의 경우 : 관리자가 컴퓨팅 서비스 오퍼링을 생성하고 Deployment Planner 필드에서 ImplicitDedicationPlanner를 선택합니다. 그런 다음 플래너 모드에서 관리자는 전용 리소스를 사용할 수 없을 때 공유 리소스의 일부 사용을 허용할 수 있는지 여부에 따라 엄격 또는 선호를 지정합니다. 사용자가 이 서비스 제공을 기반으로 VM을 만들 때마다 전용 호스트 중 하나에 할당됩니다.

## 전용 호스트를 사용하는 방법

명시적 전용 호스트를 사용하려면 명시적 전용 그룹을 사용합니다 ( "Affinity Groups" 참조 ). 예를 들어 새 VM을 만들 때 최종 사용자는 이를 전용 인프라에 배치하도록 선택할 수 있습니다. 이 작업은 일부 인프라가 사용자 계정 또는 도메인 전용으로 이미 할당된 경우에만 가능합니다.

## 전용 호스트, cluster, pod 및 zone의 동작

관리자는 원하는 경우 대상이 다른 계정/도메인으로 예약된 호스트 또는 공유되는 호스트 (특정 계정 또는 도메인 전용이 아님)에 관계없이 VM을 전용 호스트에서 라이브 마이그레이션할 수 있습니다. Mold는 경고를 생성하지만 작업은 허용됩니다.

전용 호스트는 호스트 태그와 함께 사용할 수 있습니다. 호스트 태그와 전용이 모두 요청되는 경우 VM은 두 요구 사항을 모두 충족하는 호스트에만 배치됩니다. 사용자가 요청한 호스트 태그가 있는 해당 사용자가 사용할 수 있는 전용 리소스가 없는 경우 VM이 배포되지 않습니다.

계정 또는 도메인을 삭제하면 전용이었던 모든 호스트, cluster, pod 및 zone이 해제됩니다. 이제 모든 계정 또는 도메인에서 공유하거나 관리자가 다른 계정 또는 도메인에 다시 할당하도록 선택할 수 있습니다.

시스템 VM 및 가상 라우터는 호스트 전용 동작에 영향을 줍니다. 시스템 VM 및 가상 라우터는 Mold 시스템 계정이 소유하며 모든 호스트에 배포할 수 있습니다. 그들은 명시적인 헌신을 고수하지 않습니다. 호스트에 시스템 VM 및 가상 라우터가 있으면 엄격한 암시적 전용에 적합하지 않습니다. 호스트에 특정 계정 (기본 시스템 계정)의 VM이 이미 있으므로 엄격한 암시적 전용에 호스트를 사용할 수 없습니다. 그러나 시스템 VM 또는 가상 라우터가 있는 호스트를 선호하는 암시적 전용으로 사용할 수 있습니다.

## 사용자 인증을 위해 LDAP 서버 사용

Microsoft Active Directory 또는 ApacheDS와 같은 외부 LDAP 서버를 사용하여 Mold 최종 사용자를 인증할 수 있습니다. Mold는 지정된 기본 디렉터리에서 시작하여 외부 LDAP 디렉터리 트리를 검색하고 이름, 성, 이메일 및 사용자 이름과 같은 사용자 정보를 가져옵니다.

도메인에서 계정별 자동 동기화를 구성하여 도메인의 사용자를 LDAP의 그룹 구성원으로 최신 상태로 유지할 수 있습니다.

#### Note

주의할 점은 ApacheDS는 사용자가 다른 계정으로 이동했는지 확인하는 데 필요한 가상 'memberOf' 속성을 아직 지원하지 않는다는 것입니다. Microsoft AD 및 OpenLDAP와 OpenDJ는 이를 지원합니다.

이제 LDAP 사용자를 Mold 사용자에게 연결하는 세 가지 방법이 있습니다. 이 세 가지 방법은 서로의 확장으로 개발되었습니다.

인증 위해 세 가지 경우 모두 사용자가 입력한 사용자 이름과 비밀번호가 사용됩니다.

1. manual import. 사용자는 도메인/계정에 명시적으로 매핑되고 해당 계정의 사용자로 생성됩니다.
  - a. Mold는 주어진 사용자 이름으로 사용자를 검색합니다.
  - b. 존재하는 경우 사용자가 활성화되어 있는지 확인합니다.
  - c. 사용자가 활성화된 경우 Mold는 구성된 `ldap.username.attribute` 으로 LDAP에서 검색합니다.
  - d. LDAP 사용자가 발견되면 Mold는 해당 LDAP 사용자에게 대해 반환된 보안 주체와 입력한 비밀번호로 바인드 요청을 수행합니다.
  - e. LAP의 인증 결과 확인합니다.
2. autoimport. 도메인은 해당 도메인에 아직 존재하지 않는 사용자를 가져오도록 구성됩니다. 이러한 사용자의 경우 사용자와 동일한 이름의 계정이 자동으로 생성되고 해당 계정에 사용자가 생성됩니다.
  - a. 도메인이 LDAP와 함께 사용되도록 구성된 경우
  - b. Mold는 구성된 `ldap.username.attribute` 에 의해 LDAP에서 검색합니다.
  - c. LDAP 사용자가 발견되면 Mold는 해당 LDAP 사용자에게 대해 반환된 보안 주체와 입력한 비밀번호로 바인드 요청을 수행합니다.
  - d. LDAP 인증이 체크아웃되면 Mold는 인증된 사용자가 로그인하려는 도메인에 존재하는지 확인합니다.
    - i. 사용자가 Mold에 존재하는 경우 활성화됩니다.
    - ii. 존재하지 않는 경우 사용자 이름을 계정 및 사용자 이름으로 사용하여 새 계정에 생성됩니다.
  - e. 인증이 실패할 경우 구성된 `incorrect.login.attempts.allowed` 시도 횟수 후에 사용자가 Mold에서 비활성화됩니다.
3. autosync. 도메인은 LDAP 서버를 사용하도록 구성되며 이 도메인에서는 여러 계정이 LDAP 그룹에 대해 '매핑'됩니다. 이러한 구성된 계정 중 하나에 있는 모든 사용자는 현재 LDAP 상태에 대해 확인되며 존재하는 경우 해당 LDAP 그룹에 따라 올바른 계정에 있는 것으로 간주됩니다. LDAP에 없는 경우 Mold에서 비활성화됩니다.
  - a. 도메인이 LDAP에서 사용되도록 구성된 경우
  - b. Mold는 구성된 `ldap.username.attribute` 로 LDAP에서 이를 검색합니다.
  - c. LDAP 사용자가 발견되면 매핑된 계정, 즉 LDAP 그룹이 구성된 계정의 구성원인지 확인합니다.
    - i. LDAP 사용자에게 0, 2 또는 그 이상의 멤버십이 있는 경우 계정이 비활성화되고 인증이 실패합니다.

- d. 그런 다음 Mold는 해당 LDAP 사용자에게 대해 반환된 보안 주체와 입력한 암호로 바인드 요청을 수행합니다.
- e. Mold 사용자가 없는 경우 적절한 계정에 생성됩니다.
- f. Mold 사용자가 있지만 적절한 계정에 없는 경우 자격 증명이 이동됩니다.

Mold에서 LDAP 인증을 설정하려면 Mold API 명령을 호출 `addLdapConfiguration` 하고 LDAP 서버의 호스트 이름 또는 IP 주소 및 수신 포트를 제공하십시오. 선택적으로 이 LDAP 연결이 유효한 도메인에 대해 도메인 ID를 제공할 수 있습니다. 동일한 도메인에 대해 여러 서버를 구성할 수도 있습니다. 이들은 복제본일 것으로 예상됩니다. 하나가 실패하면 다음 것이 사용됩니다.

```
cloudmonkey add ldapconfiguration hostname=localhost\
port=389\
domainid=12345678-90ab-cdef-fedc-ba0987654321
```

이것은 LDAP 사용자의 수동 가져오기를 활성화하는 데 필요한 전부입니다. `LisLdapUsers` API를 사용하여 가져올 사용자를 쿼리할 수 있습니다.

자동 가져오기 방법의 경우 Mold 도메인을 LDAP에 연결해야 합니다. 예를 들어

```
cloudmonkey link domaintoldap domainid=12345678-90ab-cdef-fedc-ba0987654321\
accounttype=2\
ldapdomain="ou=people,dc=cloudstack,dc=apache,dc=org"\
type=OU
```

자동 동기화를 사용하려는 경우 도메인이 LDAP에 연결되지 않고 하나 이상의 계정이 연결됩니다. Mold 도메인 내에서 계정을 LDAP 그룹에 연결해야 합니다. 도메인의 연결은 암시적이며 위에서 설명한 API 호출을 통해 적용해야 합니다.

```
#!/bin/bash
[ -z "$LDAP1PASSWORD" -o -z "$LDAP2PASSWORD" ] && exit 1
ROOTDOMAIN=`cloudmonkey -d json list domains name=ROOT filter=id | jq.domain[0].id`

# mapping domain and account(s) from ldap server 1
MAPPEDDOMAIN1=`cloudmonkey -d json create domain name=mappedDomain1 parentdomainid=$ROOTDOMAIN | jq.domain.id`
cloudmonkey -d json add ldapconfiguration hostname=10.1.2.5 port=389 domainid=$MAPPEDDOMAIN1
cloudmonkey -d json update configuration domainid=$MAPPEDDOMAIN1 name="ldap.basedn"
value="dc=cloudstack,dc=apache,dc=org"
cloudmonkey -d json update configuration domainid=$MAPPEDDOMAIN1 name='ldap.bind.principal'
value='cn=admin,dc=cloudstack,dc=apache,dc=org'
cloudmonkey -d json update configuration domainid=$MAPPEDDOMAIN1 name='ldap.bind.password'
value=$LDAP1PASSWORD
cloudmonkey -d json update configuration domainid=$MAPPEDDOMAIN1
name='ldap.search.group.principle' value='cn=AcsAccessGroup,dc=cloudstack,dc=apache,dc=org'
cloudmonkey -d json update configuration domainid=$MAPPEDDOMAIN1
name='ldap.user.memberof.attribute' value='memberOf'

cloudmonkey -d json ldap createaccount account='seniors' accounttype=2 domainid=$MAPPEDDOMAIN1
username=guru
cloudmonkey -d json link accounttoldap account='seniors' accounttype=2 domainid=$MAPPEDDOMAIN1
ldapdomain='cn=AcsSeniorAdmins,ou=AcsGroups,dc=cloudstack,dc=apache,dc=org' type=GROUP
cloudmonkey -d json ldap createaccount account='juniors' accounttype=0 domainid=$MAPPEDDOMAIN1
username=bystander
cloudmonkey -d json link accounttoldap account='juniors' accounttype=0 domainid=$MAPPEDDOMAIN1
ldapdomain='cn=AcsJuniorAdmins,ou=AcsGroups,dc=cloudstack,dc=apache,dc=org' type=GROUP
```

위의 예제 스크립트에 표시된 항목 외에도 다음 구성 항목을 구성할 수 있습니다 (기본값은 `openldap` 용임).

- `ldap.basedn` : LDAP 기반을 설정합니다. 예 : **OU=APAC, DC=company, DC=com**
- `ldap.bind.principal`, `ldap.bind.password` : 위의 기반으로 모든 사용자를 나열할 수 있는 사용자의 DN 및 비밀번호입니다. 예 : **CN=Administrator, OU=APAC, DC=company, DC=com**
- `ldap.user.object` : LDAP 내의 사용자 개체 유형입니다. 기본값은 AD의 경우 **user** 이고 openldap의 경우 **interorgperson** 입니다.
- `ldap.email.attribute` : 사용자에 대한 ldap 내의 이메일 속성입니다. AD 및 openldap의 기본값은 **mail** 입니다.
- `ldap.firstname.attribute` : 사용자에 대한 ldap 내의 이름 속성. AD 및 openldap의 기본값은 **givenname** 입니다.
- `ldap.lastname.attribute` : 사용자의 ldap 내의 성 속성입니다. AD 및 openldap의 기본값은 **sn** 입니다.
- `ldap.username.attribute` : LDAP 내의 사용자에 대한 사용자 이름 속성입니다. 기본값은 AD의 경우 **SAMAccountName** 이고 openldap의 경우 **uid** 입니다.

## LDAP 사용자를 그룹으로 제한

- `ldap.search.group.principle` : 선택 사항이며 설정하면 이 그룹의 사용자만 나열됩니다.

## LDAP SSL

LDAP 서버에 SSL이 필요한 경우 아래 구성을 활성화해야 합니다. LDAP용 SSL을 활성화하기 전에 LDAP 서버가 사용 중인 인증서를 가져와서 신뢰할 수 있는 키 저장소에 추가해야 합니다. 키 저장소의 경로와 암호를 알아야 합니다.

- `ldap.truststore` : 신뢰 저장소 경로
- `ldap.truststore.password` : 신뢰 저장소 비밀번호

## LDAP 그룹

- `ldap.group.object` : LDAP 내 그룹의 개체 유형입니다. 기본값은 AD의 경우 **group**이고 openldap의 경우 **groupOfUniqueNames** 입니다.
- `ldap.group.user.uniquemember` : 그룹 내의 고유 구성원에 대한 속성입니다. 기본값은 AD의 경우 **SAMAccountName**이고 openldap의 경우 **uid**입니다.

구성되면 계정 추가 페이지에 대화 상자를 열고 선택한 사용자를 가져올 수 있는 "LDAP 계정 추가" 버튼이 표시됩니다.

다음 api 명령어를 사용할 수 있습니다. `listLdapUsers` Mold에서 가져올 수 있거나 가져올 수 있는 LDAP의 사용자를 나열합니다. `ldapCreateAccount` 특정 계정에서 사용자를 수동으로 생성 `importLdapUsers` LDAP에서 사용자 일괄 가져오기

LDAP가 활성화되면 사용자는 Mold에서 직접 비밀번호를 변경할 수 없습니다.

## 사용자 인증을 위해 SAML 2.0 ID 공급자 사용

사용자 인증을 위해 Mold와 함께 SAML 2.0 ID 공급자를 사용할 수 있습니다. Mold에서 SAML 2.0 서비스 제공 업체 플러그인을 활성화해야 합니다. 첫 번째 작업을 수행하려면, 설정하여 SAML 플러그인 활성화 `saml2.enabled` 에 `true` 로 설정하고 서버를 다시 시작해야 합니다.

SAML 플러그인은 사용자가 `authorizeSamlSso` 특정 IDP에 대해 싱글 사인온을 사용하기 전에 API를 사용하여 관리자가 사용자를 인증해야 하는 인증 워크플로를 사용합니다. SAML 단일 사인온 활성화 확인란을 선택하고 사용자를 추가하거나 가져올 때 IDP를 선택하면 됩니다. 기존 사용자의 경우 관리자는 사용자 페이지로 이동하여 SAML SSO 구성 옵션을 클릭하여 사용자에게 대한 SSO를 enable/disable하고 ID 공급자를 선택할 수 있습니다. 사용자는 하나의 IDP에 대해서만 인증할 수 있습니다.

Mold 서비스 공급자 메타데이터는 `getSPMetadata` API 명령을 사용하거나 URL <http://acs-server:8080/client/api?command=getSPMetadata> 에서 액세스할 수 있습니다. 여기서 acs-server는 관리 서버의 도메인 이름 또는 IP 주소입니다. IDP 관리자는 Mold에서 SP 메타데이터를 가져와 IDP 서버에 추가할 수 있습니다.

SAML 2.0 Single Sign-On 인증을 시작하려면 로그인 페이지에서 사용자가 인증할 수 있는 ID 공급자 또는 기관/부서를 선택하고 로그인 버튼을 클릭해야 합니다. 이 작업은 `samlssso` 사용자를 아이덴티티 공급자의 로그인 페이지로 리디렉션하는 API 명령입니다. 인증이 성공하면 IDP가 사용자를 Mold로 리디렉션 합니다. 사용자가 동일한 인증된 IDP에 대해 동일한 사용자 이름 (도메인 간)을 가진 여러 사용자 계정을 가지고 있는 경우 해당 사용자는 드롭다운 목록에서 IDP 서버를 선택한 후 도메인 경로를 지정해야 합니다. 기본적으로 사용자는 도메인 경로를 지정할 필요가 없습니다. 사용자가 IDP 서버에서 성공적으로 인증되면 SAML 인증 플러그인은 사용자 이름이 SAML 인증 응답에서 반환된 사용자 이름 속성값과 일치하는 사용자 계정을 찾습니다. 특정 IDP에 대해 동일한 사용자 이름을 가진 여러 사용자 계정이 있는 경우에만 실패합니다. 그렇지 않으면 고유한 사용자 계정이 계속 진행되고 사용자가 자신의 계정에 로그인됩니다.

제한 사항 :

- 플러그인은 SAML 응답에서 IDP 서버가 반환한 사용자 속성을 사용하여 Mold에서 인증된 사용자를 찾고 매핑합니다. 기본 속성은 uid 입니다.
- SAML 인증 플러그인은 HTTP- 리디렉션 및 HTTP-Post 바인딩을 지원합니다.
- Shibboleth 2.4, SSO Circle, Microsoft ADFS, OneLogin, Feide OpenIDP, PingIdentity로 테스트되었습니다.

다음 글로벌 설정을 구성해야 합니다.

- `saml2.enabled` : SAML SSO 플러그인 사용 여부를 나타냅니다. 기본값은 **false** 입니다.
- `saml2.sp.id` : SAML2 서비스 제공 업체 식별자 문자열
- `saml2.idp.metadata.url` : SAML2 아이덴티티 공급자 메타데이터 XML URL 또는 파일 이름. URL이 제공되지 않으면 구성 디렉토리 /etc/cloudstack/management에서 파일을 찾습니다.
- `saml2.default.idpid` : IDP가 여러 개인 경우에만 사용할 기본 IDP 엔티티 ID
- `saml2.sigalg` : SAML 요청에 서명할 때 사용할 알고리즘입니다. 기본값은 SHA1이며 허용되는 알고리즘은 SHA1, SHA256, SHA384, SHA512입니다.
- `saml2.redirect.url` : 성공시 SSO가 리디렉션되어야 하는 Mold UI URL입니다. 기본값은 <http://localhost:8080/client> 입니다.
- `saml2.sp.org.name` : SAML2 서비스 제공 업체 조직 이름
- `saml2.sp.org.url` : SAML2 서비스 제공 업체 조직 URL
- `saml2.sp.contact.email` : SAML2 서비스 제공 업체 연락처 이메일 주소
- `saml2.sp.contact.person` : SAML2 서비스 제공 업체 담당자 이름
- `saml2.sp.slo.url` : SAML2 Mold 서비스 제공 업체 싱글 로그 아웃 URL
- `saml2.sp.sso.url` : SAML2 Mold 서비스 제공 업체 싱글 사인온 URL
- `saml2.user.attribute` : 사용자 이름을 포함할 SAML 응답에서 찾을 속성 이름입니다. 기본값은 **uid** 입니다.

- `saml2.timeout` : SAML2 IDP 메타데이터 새로고침 간격 (초), 최소값은 300으로 설정됩니다. 기본값은 **1800** 입니다.

# ABLESTACK Online Docs